UNIVERSITY OF BOLOGNA, CESENA CAMPUS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

February 5, 2026

Assignment #02 — Smart Drone Hangar

Course: Embedded Systems and Internet of Things

by

Nicholas Magi 0001113915
Rebecca Scarcelli 0001118585
Elettra Ventura 0001117272

Abstract: The assignment requires the development of a Smart Drone Hangar system using Arduino along with high abstract concepts such as FSM (Finite State Machines) and Task-based architecture. This document is meant to provide a detailed description about our work, how we implemented the Smart Drone Hangar (SDH) and the Drone Remote Unit (DRU) and how we made them communicate.

Contents

1 Introduction **3**

1.1 Requirements — v1.1.1-20251112 3

1.1.1 Description - Components 3

1.1.2 Description - Behaviour 4

1.1.3 Description - the deliverable 5

2 Arduino UNO Circuit **6**

3 Smart Drone Hangar **6**

3.1 Task-based architecture 6

3.1.1 BlinkingLedTask 7

3.1.2 HangarDoorTask 8

3.1.3 TemperatureMonitoringTask 8

3.1.4 HangarMainTask 9

3.1.5 UML State Diagram of the tasks FSMs 9

4 Drone Remote Unit **11**

4.1 How to run the program 11

4.1.1 Setting the Serial Port 11

1 Introduction

1.1 Requirements — v1.1.1-20251112

We want to realise an embedded system called *Smart Drone Hangar*. The prototype is meant to implement a simple smart hangar for a drone.

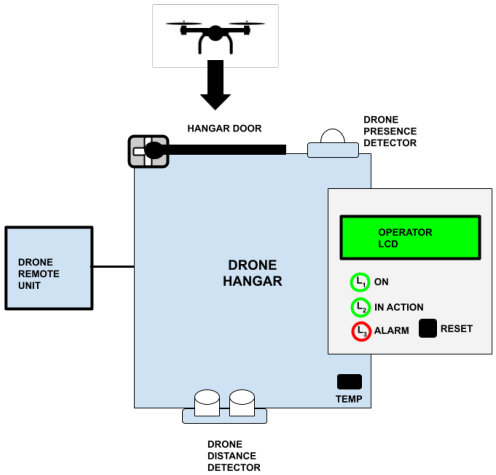


Figure 1: Suggested layout

1.1.1 Description - Components

The system is composed by two subsystems:

- **DRONE HANGAR** subsystem, based on Arduino
  - **DRONE PRESENCE DETECTOR (DPD)** is a PIR, to detect the presence of a drone near (on top) of the hangar.
  - **DRONE DISTANCE DETECTOR (DDD)** is a sonar, to measure the distance of the drone when it is inside the hangar.
  - **HANGAR DOOR (HD)** is a servo-motor, controlling the hangar door.
  - **OPERATOR LCD** is a I2C LCD, used to interact with an operator.
  - **L1** and **L2** are green leds, **L3** is a red led.

  - **RESET** is a tactile button.
  - **TEMP** is an analog temperature sensor.
- **DRONE REMOTE UNIT (DRU)** (PC-based):
  - A GUI-based subsystem that simulates a bridge to the drone to send/receive commands.
  - Communicates via **serial line** with the DRONE HANGAR subsystem.

1.1.2 Description - Behaviour

At startup, the **HD** door is closed, and the drone is assumed to be inside at rest. **L1** is ON, **L2** and **L3** are OFF, and the LCD displays `DRONE INSIDE`.

Take-off phase The drone sends an opening command via the **DRU**. Upon reception:

1. **HD** opens and the LCD displays `TAKE OFF`.
2. The system monitors **DDD**.
3. When measured distance $> D1$ for more than $T1$ seconds, the drone is assumed to have exited.
4. **HD** closes and the LCD displays `DRONE OUT`.

Landing phase The drone sends an opening command via **DRU**.

1. If **DPD** detects the drone, **HD** opens and the LCD displays `LANDING`.
2. The system monitors **DDD**.
3. When distance $< D2$ for more than $T2$ seconds, the drone is assumed to have landed.
4. **HD** closes and the LCD displays `DRONE INSIDE`.

During the take-off and landing phases, **L2** blinks, with period 0.5 second – otherwise it is off.

Temperature monitoring is active whenever the drone is inside or in transition:

- **Pre-alarm**: If $Temp \geq Temp1$ for $> T3$ seconds, new operations are suspended. Ongoing operations are allowed to finish. The system recovers if temperature drops below $Temp1$.
- **Alarm**: If $Temp \geq Temp2$ for $> T4$ seconds (where $Temp2 > Temp1$):
  1. **HD** is closed immediately.
  2. **L3** turns ON and LCD displays `ALARM`.
  3. If the drone is outside, an `ALARM` message is sent via **DRU**.
  4. All operations are suspended until the **RESET** button is pressed.

The value of the `D1`, `D2`, `T1`, `T2`, `T3`, `T4`, `Temp1`, `Temp2` parameters is not fixed/specified (to be chosen, useful for prototyping/testing the system).

Develop the software using a **task-based architecture** and **synchronous Finite State Machines (FSM)** for the Arduino part. The PC part (DRU) must feature a GUI to:

- Send simulation commands (Take-off/Landing).
- Visualize drone state (Rest, Taking off, Operating, Landing).
- Visualize hangar state (Normal, Alarm).
- Display real-time distance during landing.

For any aspect not specified, you are free to choose the approach you consider more useful or valuable.

1.1.3 Description - the deliverable

The deliverable consists in a zipped folder `assignment-02.zip` including three directories:

- `drone-hangar`, including the source code of the Arduino-based subsystem
- `drone-remote-unit`, including the source code of the PC-based part
- `doc`, including: a brief **report** describing the solution, in particular the diagram of the FSMs and the representation of the schema/breadboard – using tools such as TinkerCad or Fritzing , and a **short video** (or the link to a video on the UNIBO OneDrive cloud) demonstrating the system.

2 Arduino UNO Circuit

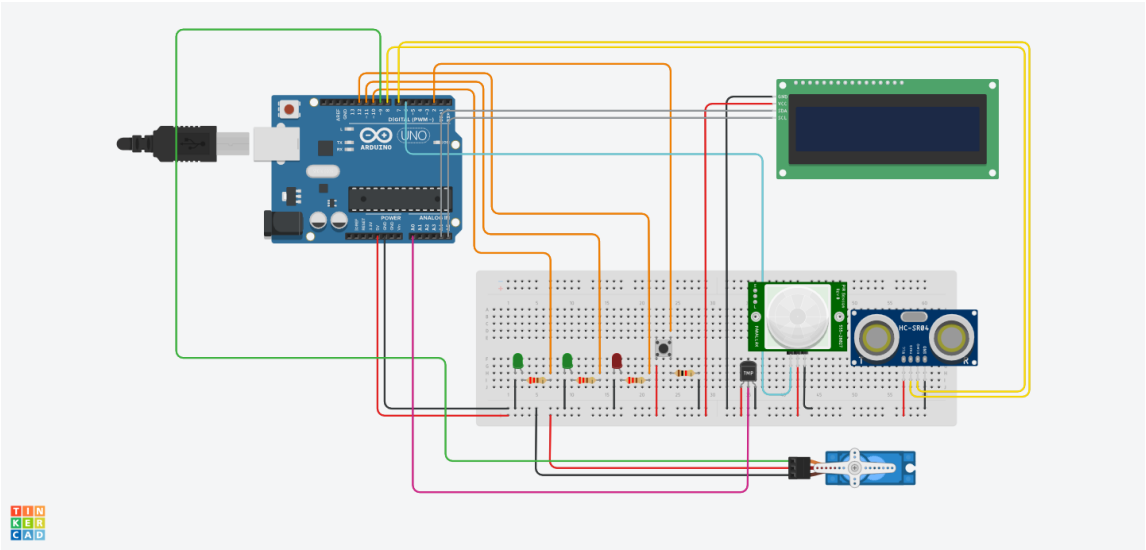


Figure 2: Screenshot of the circuit made with TinkerCad

3 Smart Drone Hangar

3.1 Task-based architecture

When looking at the assignment description, our first focus was to try to define the tasks hidden in it — in order to decompose and modularize the system. Much attention was put on the Smart Drone Hangar subsystem, since it is designed to be implemented using a **task-based architecture**.

From a theoretical point of view, we can recall that a task is defined as a *single well-defined unit of work/job to be done*, and that the behaviour of each task can be represented by a **finite-state machine**.

A **state machine** is no other than a general model of a system with discrete dynamics that at *each reaction maps valuation of the inputs to valuations of the outputs, where the map may depend on its current state*.

A **finite-state machine** is a state machine whose set of possible states is finite (*i.e. has a finite number of elements*).

Considering all of that, we defined the following tasks:

1. `BlinkingLedTask`
2. `HangarDoorTask`
3. `TemperatureMonitoringTask`
4. `HangarMainTask`

In order to orchestrate the progress of tasks as time goes on, a **scheduler** is mandatory. Following the Round Robin scheduling technique, a period of time (*called "quantum"*) is assigned to each task. During this period of time the task can use the CPU. If the task finishes its job before the ending of its quantum, it should leave the system. Otherwise, it goes back into the queue of tasks to be executed. It's up to the scheduler to choose the order of tasks to execute, regardless of their priority.

The tricky challenge is to choose a quantum of time **not too small** to make the system unresponsive and loose events and **not too big** to incur in overrun exceptions.

3.1.1 BlinkingLedTask

Hardware needed

- 1x LED

Quantum 0.5 seconds

Goal The meaning of this task — as the name suggests — is to handle the blinking of LED `L2` during take-offs and landings of the drone. It needs to read the state of the drone in order to decide whether to start blinking or not.

3.1.2 HangarDoorTask

Hardware needed

- 1x Servo motor

Quantum 0.5 seconds

Goal The task that opens and closes the hangar door (represented by the Servo motor). It reads the drone state in order to start opening the door, and the hangar state to understand wheter the system is in alarm and thus close the door. It is responsible for changing the **isDoorOpen**, one of the shared variables used to let tasks communicate.

3.1.3 TemperatureMonitoringTask

Hardware needed

- 1x Temperature sensor
- 1x Button (Reset button)
- 1x LED
- 1x LCD

Quantum 0.5 seconds

Goal It reads the drone state and understands in what moments it should check if the heat inside the hangar exceeds some thresholds. It is responsible for changing the **hangar state**, one of the three shared variables used to let tasks communicate.

3.1.4 HangarMainTask

Hardware needed

- 1x LED
- 1x Sonar (Drone Distance Detector)
- 1x Pir (Drone Presence Detector)
- 1x LCD

Quantum 0.5 seconds

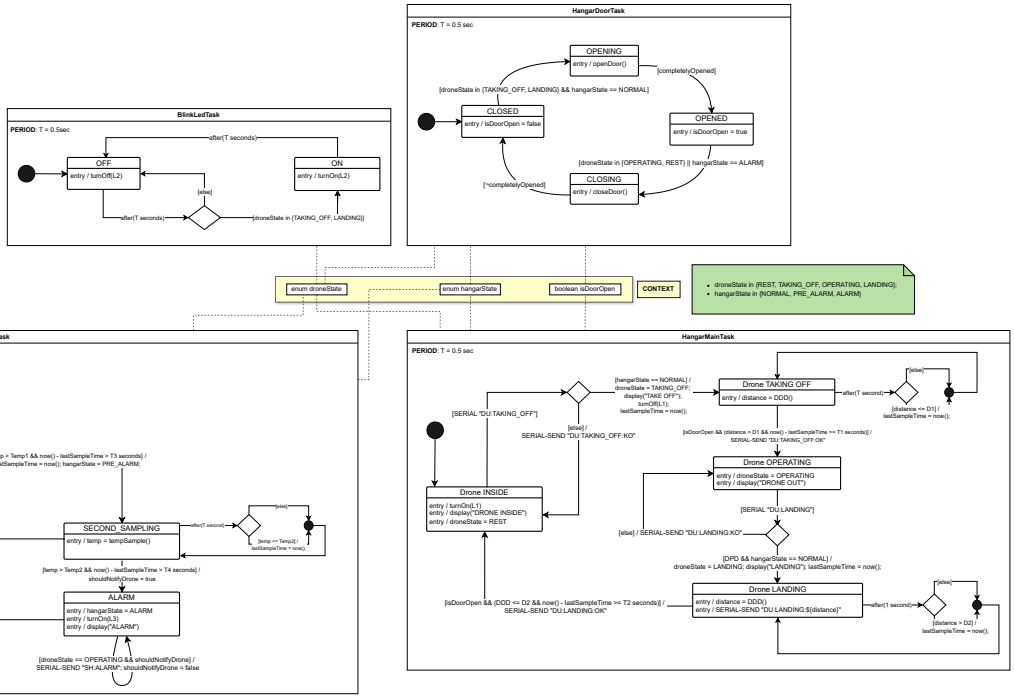
Goal The most complex task of all. It relates directly to the drone, understanding its position during time (inside or outside). It is responsible for changing the **drone state**, one of the three shared variables used to let tasks communicate.

3.1.5 UML State Diagram of the tasks FSMs

Smart Hangar Finite State Machines

UML State Diagram

Nicholas Magi	0001113915	nicholas.magi@studio.unibo.it
Rebecca Scarcelli	0001118585	rebecca.scarcelli@studio.unibo.it
Elettra Ventura	0001117272	elettra.ventura@studio.unibo.it



4 Drone Remote Unit

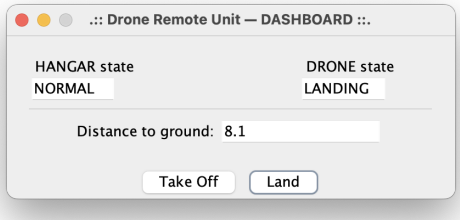


Figure 3: Screenshot of the GUI

4.1 How to run the program

In order to start and work properly, the application needs **Arduino to be connected to serial port**. Default port is set to `/dev/cu.usbmodem1201`.

To run the application, simply type in your terminal:

- on UNIX OS:

```
./gradlew run
```

- on Windows OS:

```
./gradlew.bat run
```

4.1.1 Setting the Serial Port

To change the default serial port value, specify the new port as argument to gradlew with the command:

- on UNIX OS:

```
./gradlew run --args="my/serial/port"
```

- on Windows OS:

```
./gradlew.bat run --args="my/serial/port"
```